

Q&A — Train/Test Split Methods

Dataset used throughout: `data/auto.csv` (predicting `mpg` from `horsepower`)

Method 1 — Simple Random Train/Test Split (Validation Set Approach)

Split the data randomly into 70% training and 30% test. Fit a linear regression and compute MSE.

```
import pandas as pd
import numpy as np
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_squared_error

df = pd.read_csv('data/auto.csv', na_values='?').dropna()
X = df[['horsepower']].values
y = df['mpg'].values

X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.3,          # 30% test
    random_state=42         # for reproducibility
)

model = LinearRegression().fit(X_train, y_train)
mse = mean_squared_error(y_test, model.predict(X_test))
print(f"Train size: {len(X_train)}, Test size: {len(X_test)}")
print(f"Test MSE: {mse:.2f}")
```

When to use: Quick baseline. Easy to implement. Drawback: result depends heavily on which rows land in train vs test (high variance).

Method 2 — Manual 50/50 Split by Row Sampling

Sample exactly half the rows as training, use the rest as test. (Matches the textbook approach.)

```
import pandas as pd
import numpy as np
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error

df = pd.read_csv('data/auto.csv', na_values='?').dropna()

train_df = df.sample(196, random_state=1)          # sample 196 rows
test_df = df[~df.index.isin(train_df.index)]      # remainder
```

```

X_train = train_df[['horsepower']].values
y_train = train_df['mpg'].values
X_test  = test_df[['horsepower']].values
y_test  = test_df['mpg'].values

model = LinearRegression().fit(X_train, y_train)
mse = mean_squared_error(y_test, model.predict(X_test))
print(f"Train: {len(X_train)}, Test: {len(X_test)}")
print(f"Test MSE: {mse:.2f}")

```

When to use: Mirrors the ISLR textbook validation-set approach. Same idea as `train_test_split` but uses `.sample()` directly on a DataFrame.

Method 3 — Condition-Based Split (e.g., by Year)

Split based on a column value — e.g., train on years before 2005, test on 2005 onwards. Common in time-series or structured data.

```

import pandas as pd
import numpy as np
from sklearn.discriminant_analysis import LinearDiscriminantAnalysis
from sklearn.metrics import accuracy_score

df = pd.read_csv('data/smarket.csv')
df['Up'] = (df['Direction'] == 'Up').astype(int)

X = df[['Lag1', 'Lag2']].values
y = df['Up'].values

train_mask = df['Year'].values < 2005 # boolean mask
X_train, y_train = X[train_mask], y[train_mask]
X_test, y_test = X[~train_mask], y[~train_mask]

model = LinearDiscriminantAnalysis().fit(X_train, y_train)
print(f"Train: {len(X_train)}, Test: {len(X_test)}")
print(f"Test Accuracy: {accuracy_score(y_test, model.predict(X_test)):.4f}")

```

When to use: Time series or any scenario where future data must not leak into training. Prevents data leakage from future observations.

Method 4 — K-Fold Cross-Validation (k=10)

Split the data into 10 equal folds. Train on 9, test on 1, rotate. Average the 10 MSE scores.

```

import pandas as pd
import numpy as np
from sklearn.linear_model import LinearRegression

```

```

from sklearn.model_selection import KFold, cross_val_score

df = pd.read_csv('data/auto.csv', na_values='?').dropna()
X = df[['horsepower']].values
y = df['mpg'].values

kf = KFold(n_splits=10, shuffle=True, random_state=1)

model = LinearRegression()
scores = cross_val_score(model, X, y,
                          cv=kf,
                          scoring='neg_mean_squared_error')

print(f"10-Fold CV MSE scores: {-scores.round(2)}")
print(f"Mean MSE: {-scores.mean():.2f}")
print(f"Std:      {scores.std():.2f}")

```

When to use: Standard choice. Balances bias-variance in the error estimate. Much faster than LOOCV. The `shuffle=True` avoids bias from ordered data.

Method 5 — LOOCV (Leave-One-Out Cross-Validation)

Each observation is left out once as the test set. N models are trained in total.

```

import pandas as pd
import numpy as np
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import KFold, cross_val_score

df = pd.read_csv('data/auto.csv', na_values='?').dropna()
X = df[['horsepower']].values
y = df['mpg'].values

# LOOCV: n_splits = number of observations
loocv = KFold(n_splits=len(X), shuffle=False)

model = LinearRegression()
scores = cross_val_score(model, X, y,
                          cv=loocv,
                          scoring='neg_mean_squared_error',
                          n_jobs=-1)

print(f"LOOCV MSE: {-scores.mean():.2f}")

```

When to use: When the dataset is small and you want the least biased estimate of test error. Very slow on large datasets (trains N models). For linear models, a shortcut formula exists that avoids refitting N times.

Method 6 — Stratified K-Fold (for Classification)

Preserves class proportions in each fold — critical when classes are imbalanced.

```
import pandas as pd
import numpy as np
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import StratifiedKFold, cross_val_score

df = pd.read_csv('data/default.csv')
df['default_yes'] = (df['default'] == 'Yes').astype(int)

X = df[['balance', 'income']].values
y = df['default_yes'].values

skf = StratifiedKFold(n_splits=5, shuffle=True, random_state=1)

model = LogisticRegression(max_iter=1000)
scores = cross_val_score(model, X, y,
                          cv=skf,
                          scoring='accuracy')

print(f"Stratified 5-Fold Accuracy scores: {scores.round(4)}")
print(f"Mean Accuracy: {scores.mean():.4f}")

# Check class balance is preserved
for fold, (tr, te) in enumerate(skf.split(X, y)):
    print(f"Fold {fold+1} — Test class 1 rate: {y[te].mean():.3f}")
```

When to use: Classification with imbalanced classes (e.g., only 3% defaults). Regular KFold might give a fold with zero positive examples — StratifiedKFold guarantees proportional representation.

Method 7 — cross_val_predict (Get Predictions for the Whole Dataset)

Instead of just scores, get the out-of-fold predicted label for every observation.

```
import pandas as pd
import numpy as np
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import cross_val_predict
from sklearn.metrics import confusion_matrix, classification_report

df = pd.read_csv('data/default.csv')
df['default_yes'] = (df['default'] == 'Yes').astype(int)

X = df[['balance', 'income']].values
y = df['default_yes'].values

model = LogisticRegression(max_iter=1000)

# Each row's prediction comes from a fold that did NOT train on it
```

```
y_pred = cross_val_predict(model, X, y, cv=5)

print("Confusion Matrix:\n", confusion_matrix(y, y_pred))
print("\nClassification Report:\n", classification_report(y, y_pred))
```

When to use: When you need row-level predictions for the entire dataset (e.g., to build a confusion matrix across all CV folds). Useful for imbalanced evaluation without a separate test set.

Summary Table

Method	sklearn call	Key params	Best for
Random split	<code>train_test_split</code>	<code>test_size</code> , <code>random_state</code>	Quick baseline
Sample split	<code>df.sample(n)</code>	<code>random_state</code>	Textbook approach
Condition split	Boolean mask <code>df['Year'] < X</code>	—	Time series / structured
K-Fold CV	<code>KFold(n_splits=10)</code>	<code>shuffle=True</code>	General purpose
LOOCV	<code>KFold(n_splits=len(X))</code>	—	Small datasets
Stratified K-Fold	<code>StratifiedKFold(n_splits=5)</code>	<code>shuffle=True</code>	Imbalanced classes
CV predictions	<code>cross_val_predict</code>	<code>cv=5</code>	Full-dataset evaluation